1. **Watch the Terraform-05 video.**

   Completed

2. **Execute the script shown in the video.**

Implementation Steps

1. Created a project folder named task-2 inside C:\terraform-05.
2. Created a main.tf file containing the AWS provider and EC2 instance configuration.
3. Configured the AWS provider to use the us-east-1 region.
4. Created an EC2 instance using the t3.micro instance type and the existing key pair mytest_key.
5. Used the local-exec provisioner to execute a local command after the EC2 instance was created.
6. Retrieved the EC2 public IP address using self.public_ip and stored it in a local file named ip.txt.
7. Implemented a destroy-time provisioner using when = destroy to execute a command when the EC2 instance was destroyed.
8. Used the destroy provisioner to create a file named destroy.txt containing a destruction message.
9. Implemented provisioner failure handling using on_failure = fail and tested it with a command that returned an error.
10. Implemented on_failure = continue to allow Terraform to continue execution even when a provisioner command failed.
11. Practised Terraform debugging by enabling logs using the TF_LOG environment variable.
12. Used TF_LOG_PATH to store Terraform debugging information in a log file.
13. Practised resource tainting to mark a resource for recreation.
14. Verified that Terraform planned to replace the tainted resource.
15. Used the untaint operation to remove the tainted status from a resource.
16. Practised importing an existing AWS security group into Terraform state.
17. Verified the imported resource using Terraform state commands.

18. Destroyed the resources created for the practice and verified that the Terraform state was empty.

EC2 instance with provisioners:

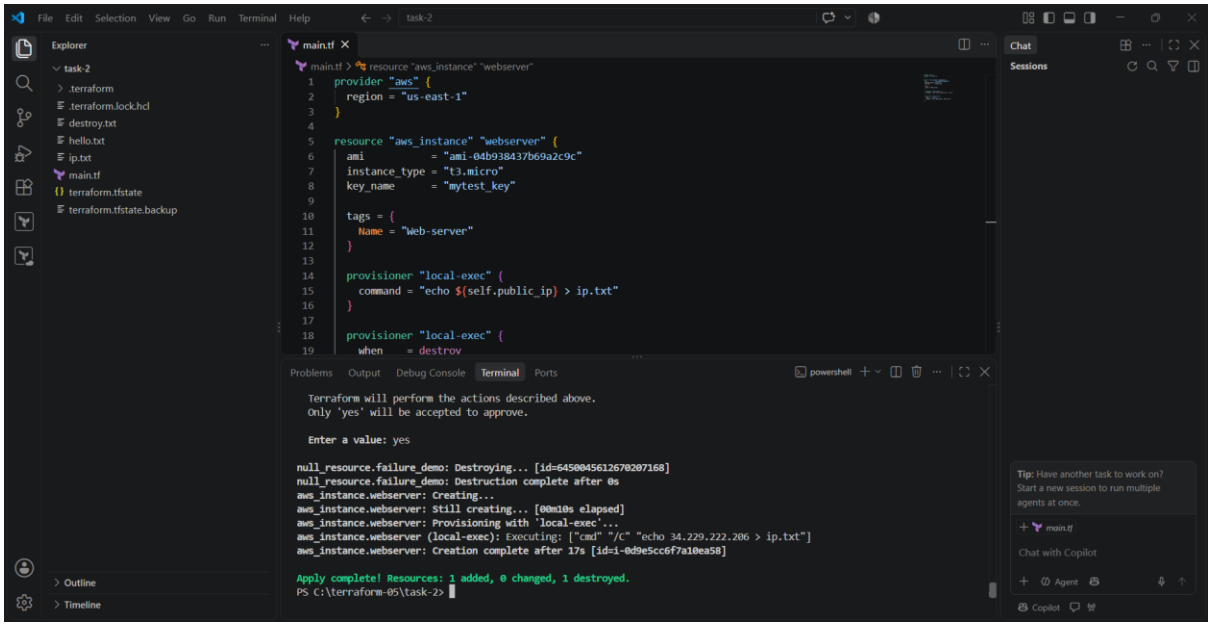
```
provider "aws" {
 region = "us-east-1"
}

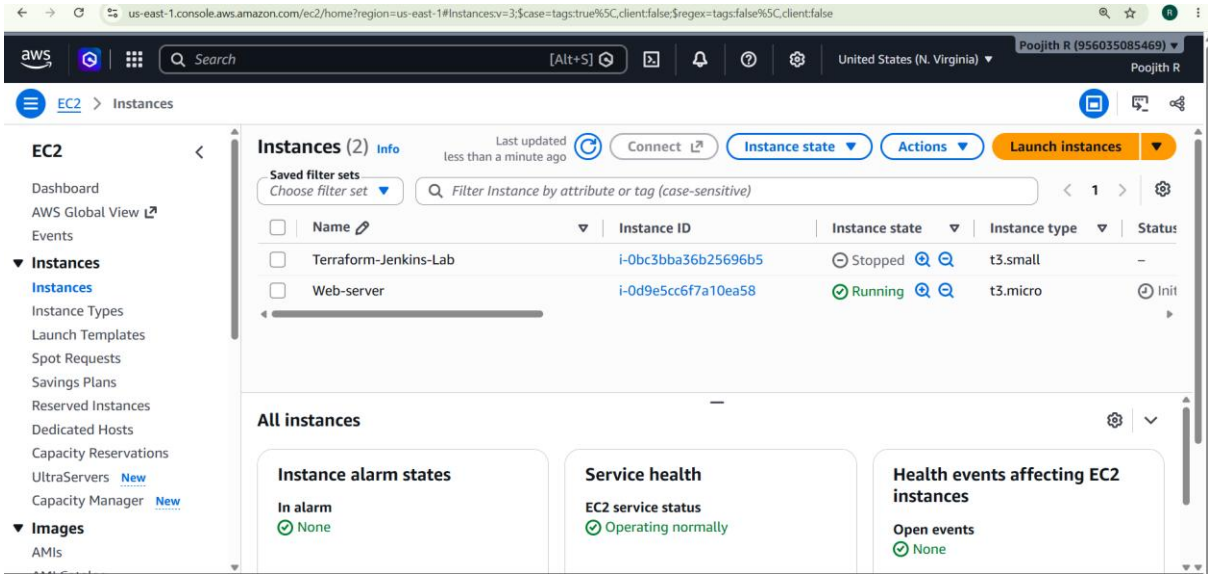
resource "aws_instance" "web" {
 ami           = "ami-04b938437b69a2c9c"
 instance_type = "t3.micro"
 key_name      = "mytest_key"

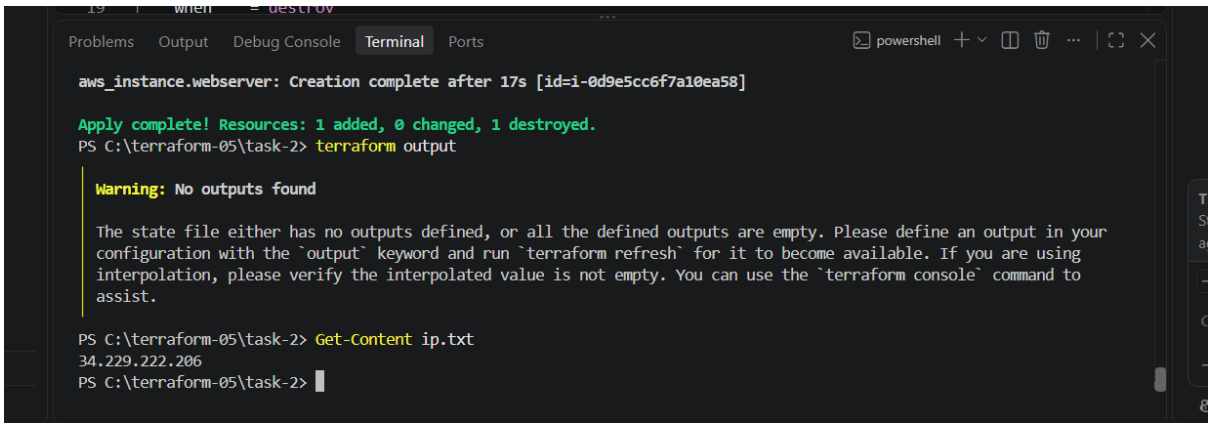
 provisioner "local-exec" {
  command = "echo ${self.public_ip} > ip.txt"
 }

 provisioner "local-exec" {
  when    = destroy
  command = "echo destroyed > destroy.txt"
 }

 tags = {
  Name = "Web-server"
 }
}
```





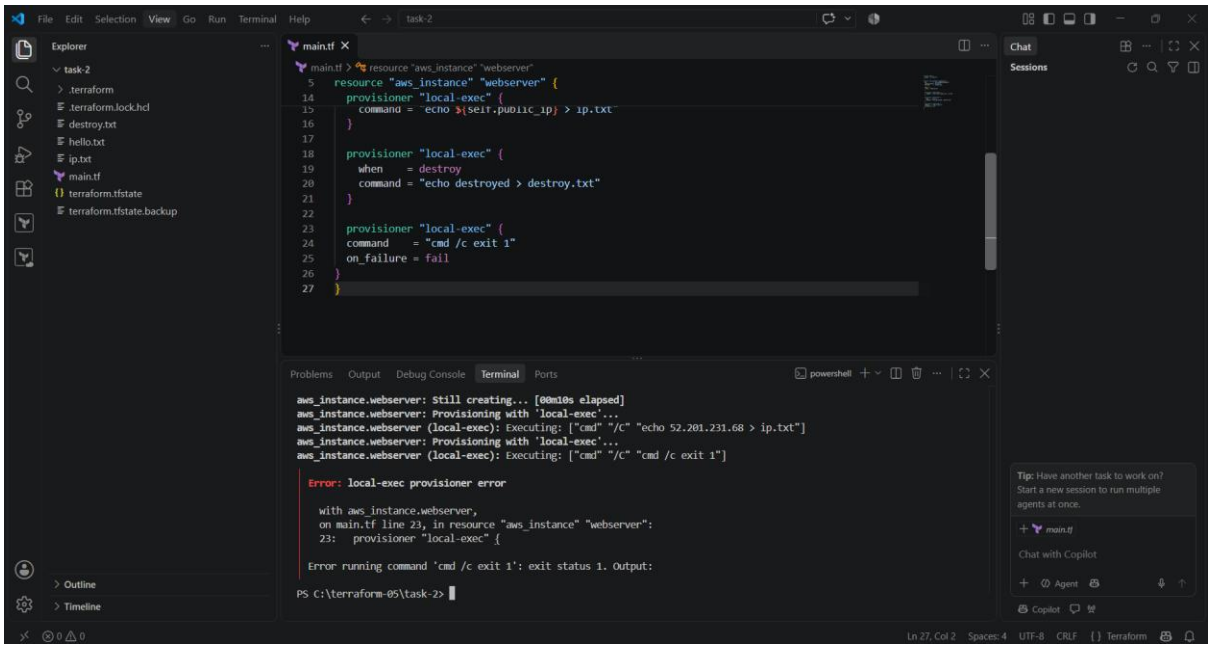


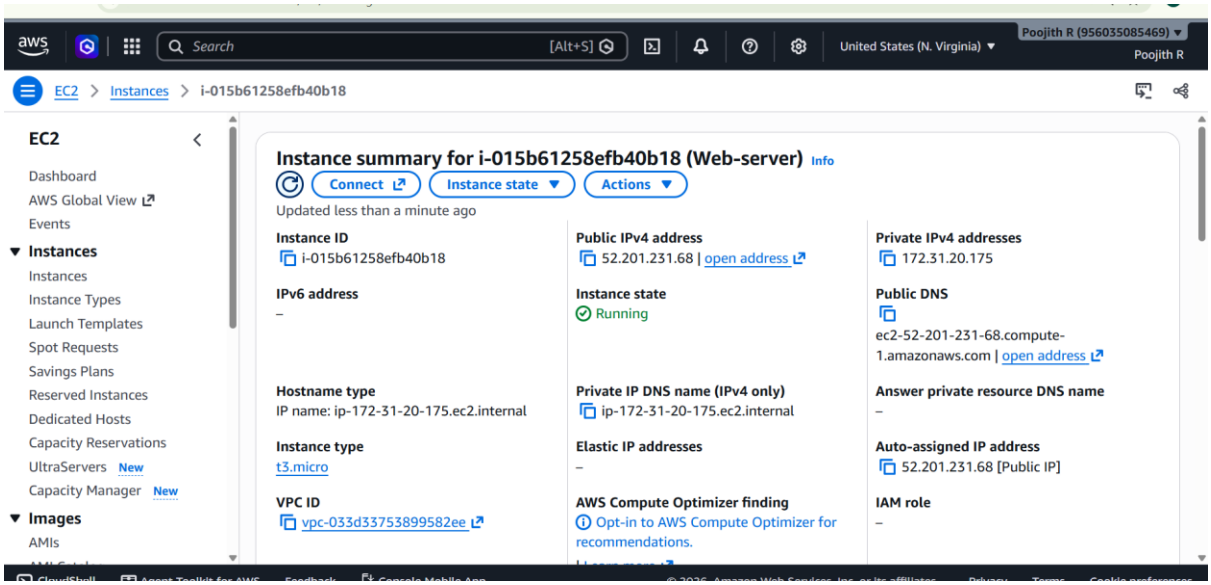
On_failure = fail

```
provisioner "local-exec" {
  command    = "cmd /c exit 1"
```

on_failure = fail

}





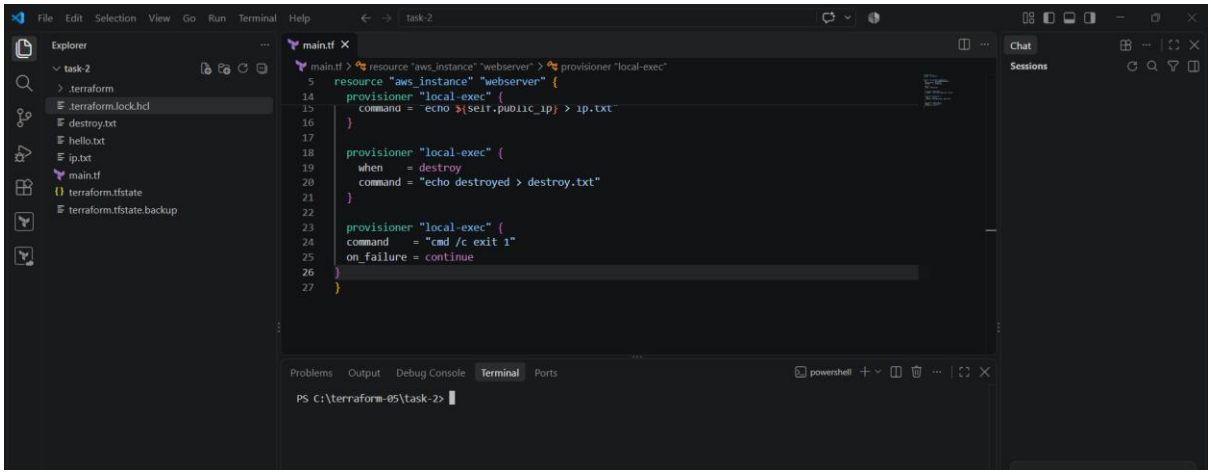
on_failure = continue

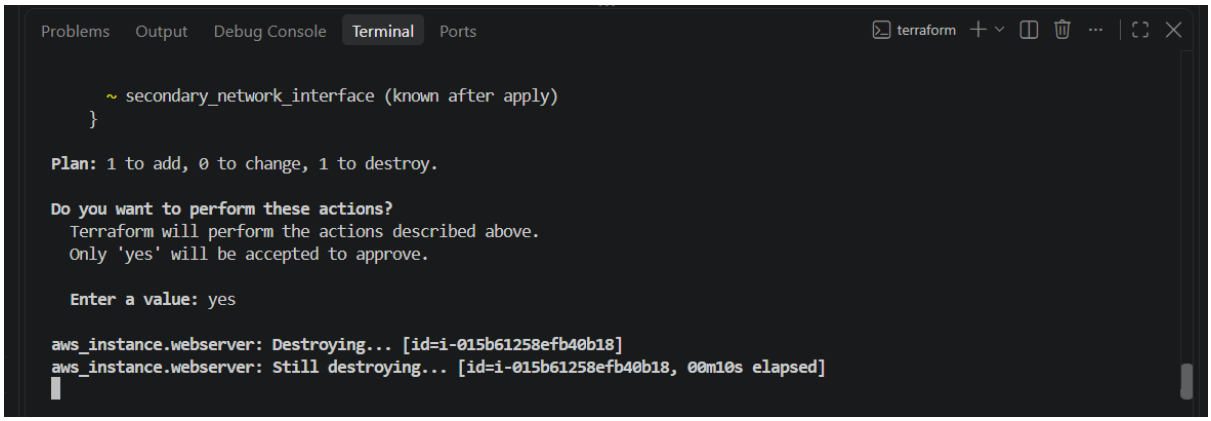
provisioner "local-exec" {

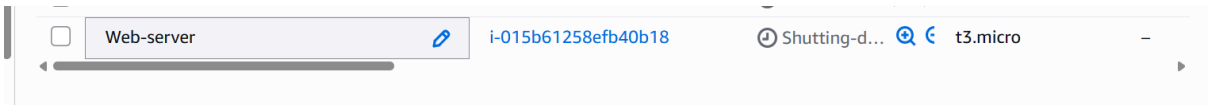
command = "cmd /c exit 1"

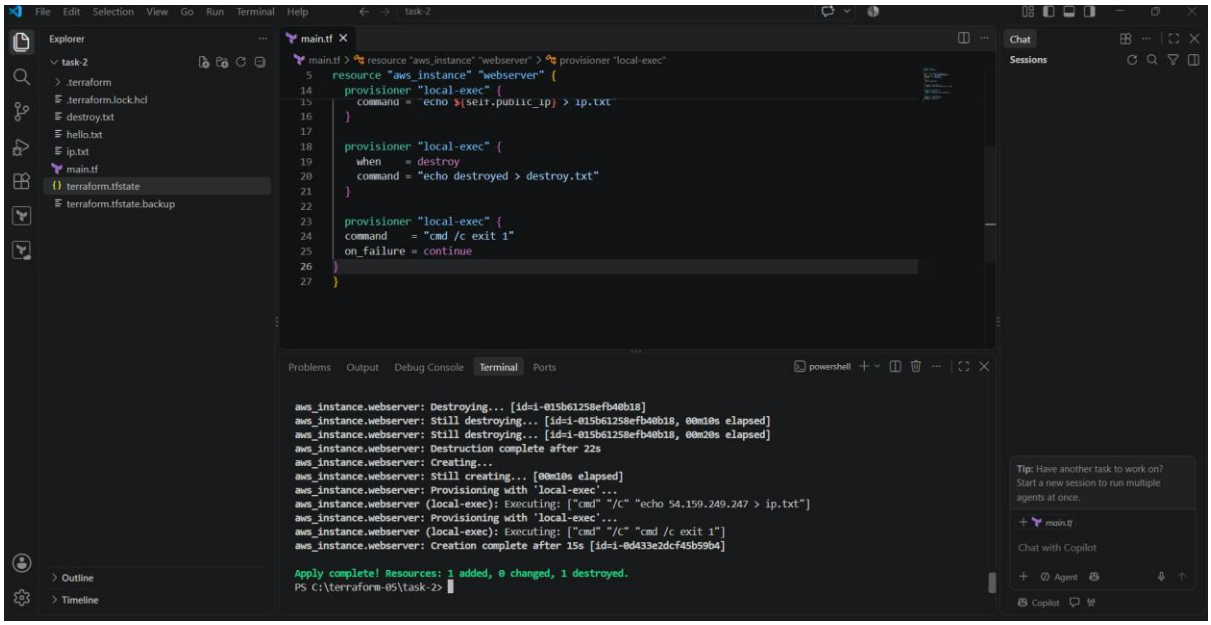
on_failure = continue

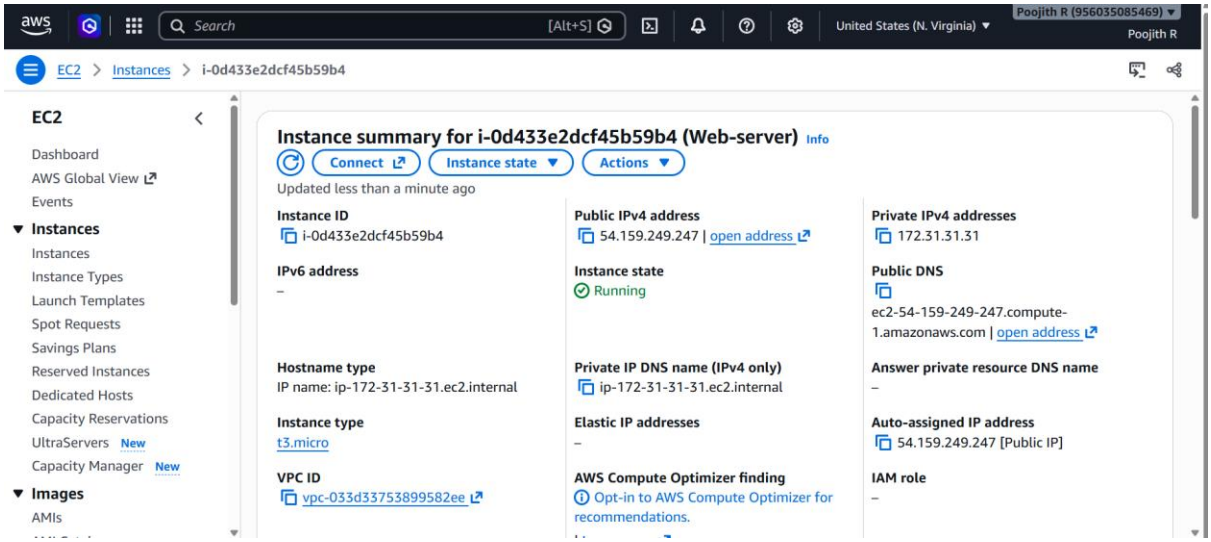
}


File Edit Selection View Go Run Terminal Help
task-2
Explorer
task-2
.terraform
.terraform.lock.hcl
destroy.txt
hello.txt
ip.txt
main.tf
terraform.tfstate
terraform.tfstate.backup
main.tf
resource "aws_instance" "webserver" {
provisioner "local-exec" {
command = "echo ${self.public_ip} > ip.txt"
}
provisioner "local-exec" {
when = destroy
command = "echo destroyed > destroy.txt"
}
provisioner "local-exec" {
command = "cmd /c exit 1"
on_failure = continue
}
}
Chat
Sessions
Problems Output Debug Console Terminal Ports
powershell
PS C:\terraform-05\task-2>


Problems Output Debug Console Terminal Ports
terraform
~ secondary_network_interface (known after apply)
}
Plan: 1 to add, 0 to change, 1 to destroy.
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
aws_instance.webserver: Destroying... [id=i-015b61258efb40b18]
aws_instance.webserver: Still destroying... [id=i-015b61258efb40b18, 00m10s elapsed]


Web-server
i-015b61258efb40b18
Shutting-d...
t3.micro
–

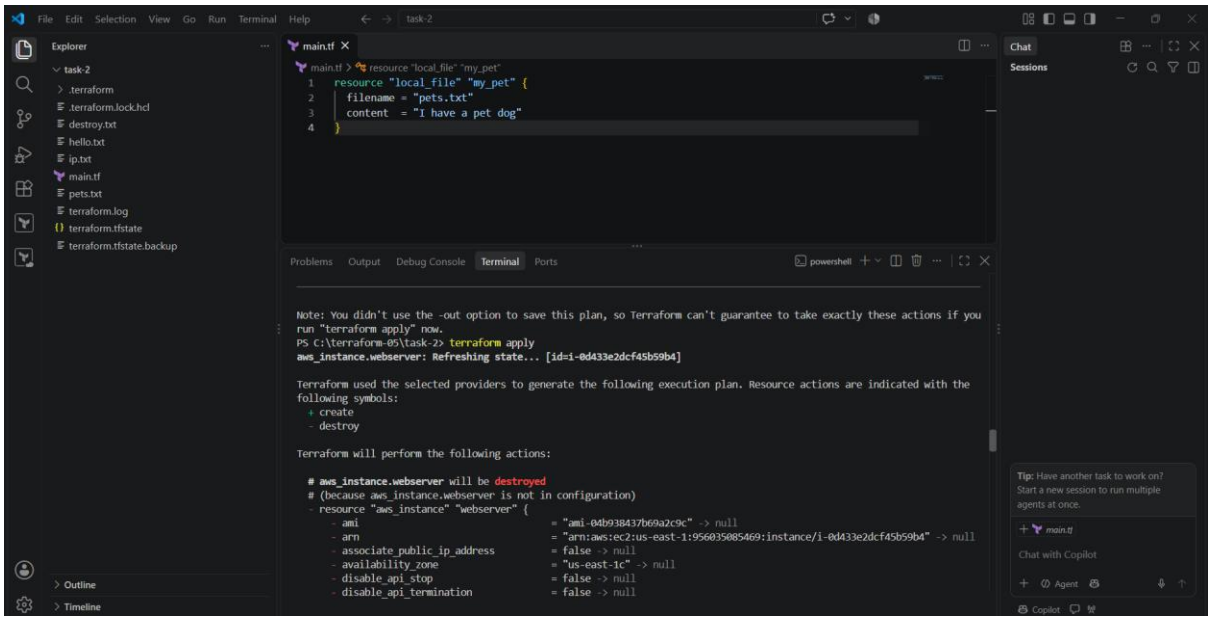

File Edit Selection View Go Run Terminal Help
task-2
Explorer
task-2
.terraform
.terraform.lock.hcl
destroy.txt
hello.txt
ip.txt
main.tf
terraform.tfstate
terraform.tfstate.backup
main.tf
resource "aws_instance" "webserver" {
provisioner "local-exec" {
command = "echo ${self.public_ip} > ip.txt"
}
provisioner "local-exec" {
when = destroy
command = "echo destroyed > destroy.txt"
}
provisioner "local-exec" {
command = "cmd /c exit 1"
on_failure = continue
}
}
Chat
Sessions
Problems Output Debug Console Terminal Ports
powershell
aws_instance.webserver: Destroying... [id=i-015b61258efb40b18]
aws_instance.webserver: Still destroying... [id=i-015b61258efb40b18, 00m10s elapsed]
aws_instance.webserver: Still destroying... [id=i-015b61258efb40b18, 00m20s elapsed]
aws_instance.webserver: Destruction complete after 22s
aws_instance.webserver: Creating...
aws_instance.webserver: Still creating... [00m10s elapsed]
aws_instance.webserver: Provisioning with 'local-exec'...
aws_instance.webserver (local-exec): Executing: ["cmd" "/C" "echo 54.159.249.247 > ip.txt"]
aws_instance.webserver: Provisioning with 'local-exec'...
aws_instance.webserver (local-exec): Executing: ["cmd" "/C" "cmd /c exit 1"]
aws_instance.webserver: Creation complete after 15s [id=i-0d433e2dcf45b59b4]
Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
PS C:\terraform-05\task-2>
Outline
Timeline
Tip: Have another task to work on? Start a new session to run multiple agents at once.
Chat with Copilot
Agent
Copilot

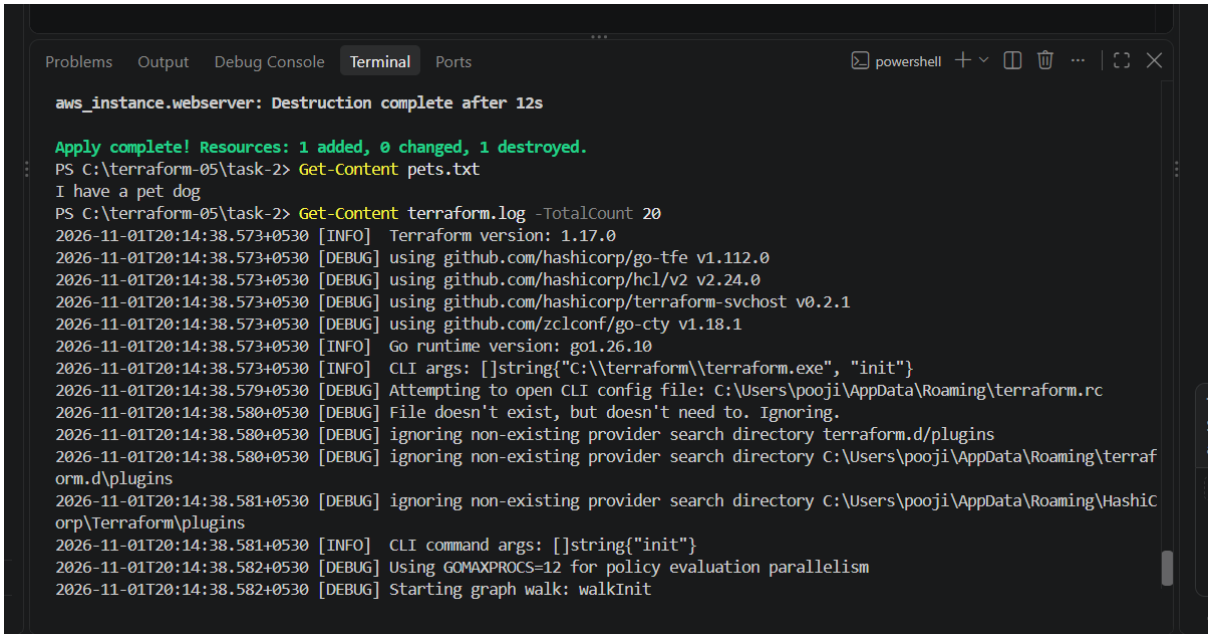


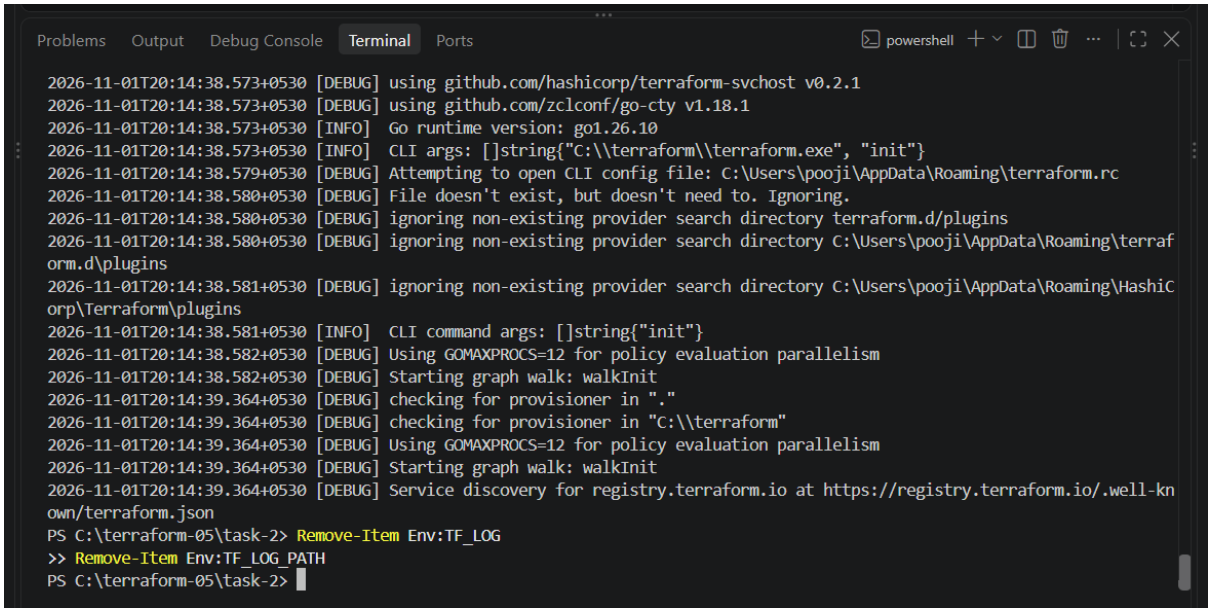
Debug:

$env:TF_LOG = "DEBUG"

$env:TF_LOG_PATH = "terraform.log"

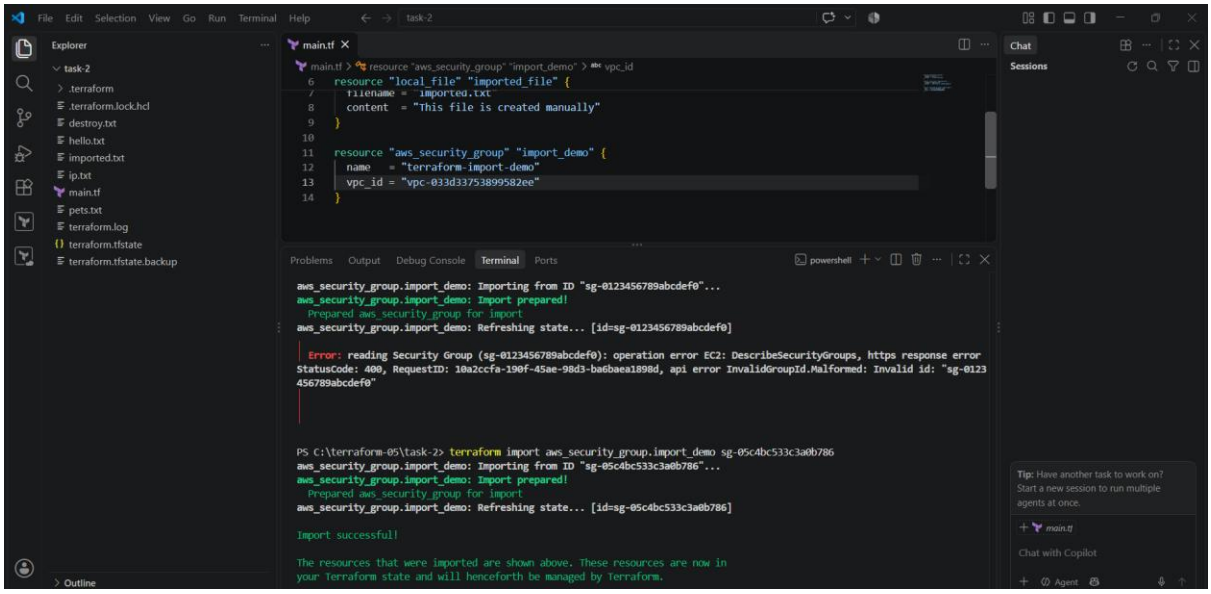






Import:

terraform import aws_security_group.import_demo sg-05c4bc533c3a0b786



3. Create one EC2 instance with httpd installed using a Terraform script.

Implementation Steps

1. Installed Terraform on the local Windows machine and configured AWS credentials.
2. Created a project folder named terraform-05 and a subfolder named task-3.
3. Created a main.tf file containing the AWS provider, default VPC data source, security group, EC2 instance, and output configurations.
4. Configured the AWS provider to use the us-east-1 region.
5. Retrieved the default VPC using a Terraform data source.
6. Created a security group allowing inbound HTTP traffic on port 80 and outbound traffic to all destinations.
7. Created an EC2 instance using the t3.micro instance type and the existing key pair mytest_key.
8. Used the EC2 user_data script to install Apache (httpd), enable and start the service, and create a custom HTML page.
9. Added the instance name tag as Terraform-Apache-WebServer.
10. Configured Terraform outputs to display the EC2 public IP address and website URL.
11. Executed terraform init to initialize the working directory.
12. Executed terraform validate to validate the Terraform configuration.

13. Executed terraform plan to review the infrastructure changes.
14. Executed terraform apply and confirmed the deployment.
15. Retrieved the public IP address and website URL from Terraform outputs.
16. Opened the website URL in a browser and verified that the custom Apache webpage was displayed successfully.

Code – main.tf

```
provider "aws" {
  region = "us-east-1"
}

data "aws_vpc" "default" {
  default = true
}

resource "aws_security_group" "web_sg" {
  name        = "terraform-web-sg"
  description = "Allow HTTP traffic"
  vpc_id      = data.aws_vpc.default.id

  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
```

```
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_instance" "webserver" {
  ami                    = "ami-04b938437b69a2c9c"
  instance_type          = "t3.micro"
  key_name               = "mytest_key"
  vpc_security_group_ids = [aws_security_group.web_sg.id]

  user_data = <<-EOF
    #!/bin/bash
    yum install -y httpd
    systemctl start httpd
    systemctl enable httpd
    echo "<h1>Welcome to Terraform Apache Web Server</h1>" >
/var/www/html/index.html
  EOF

  tags = {
    Name = "Terraform-Apache-WebServer"
  }
}

output "public_ip" {
  value = aws_instance.webserver.public_ip
```

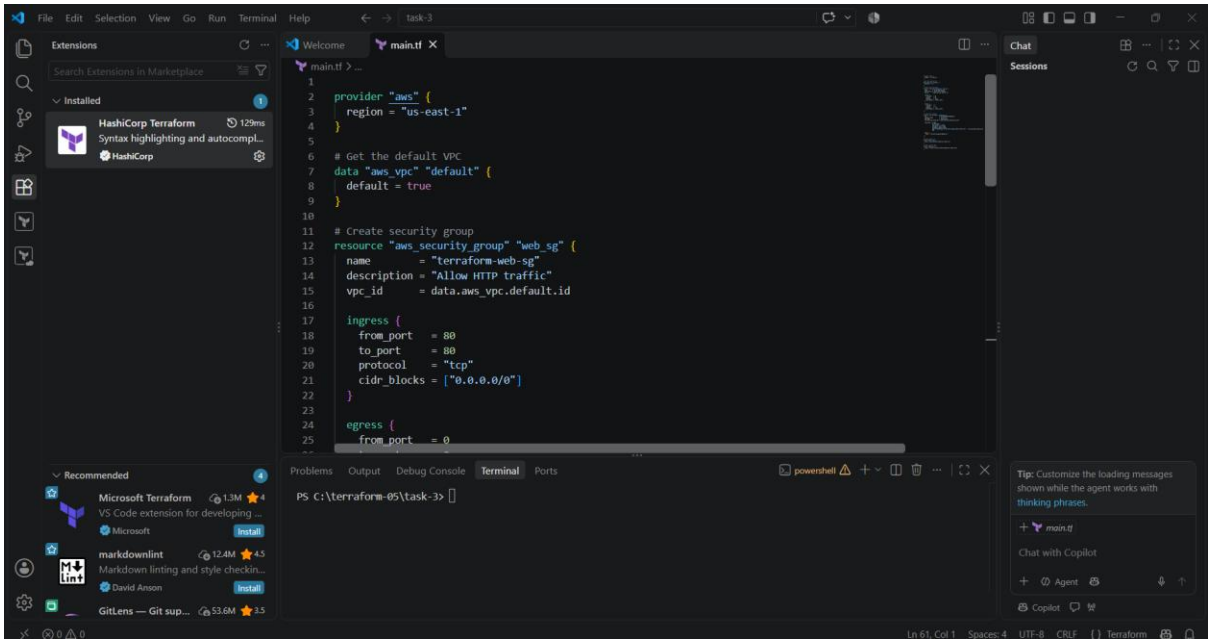
}

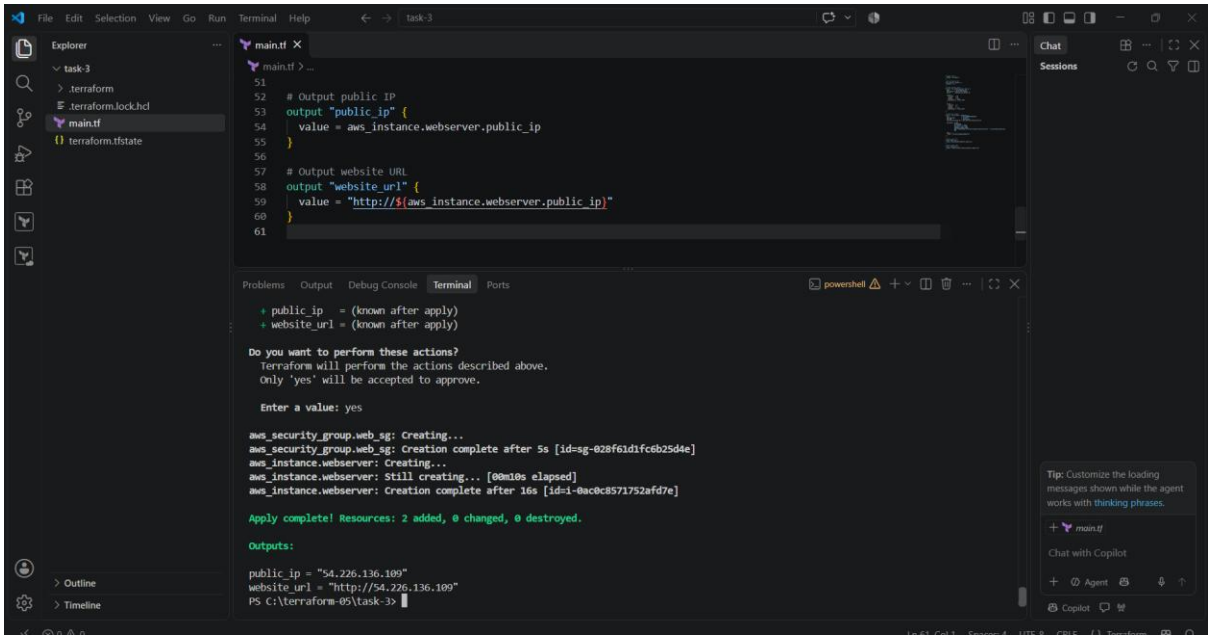
output "website_url" {

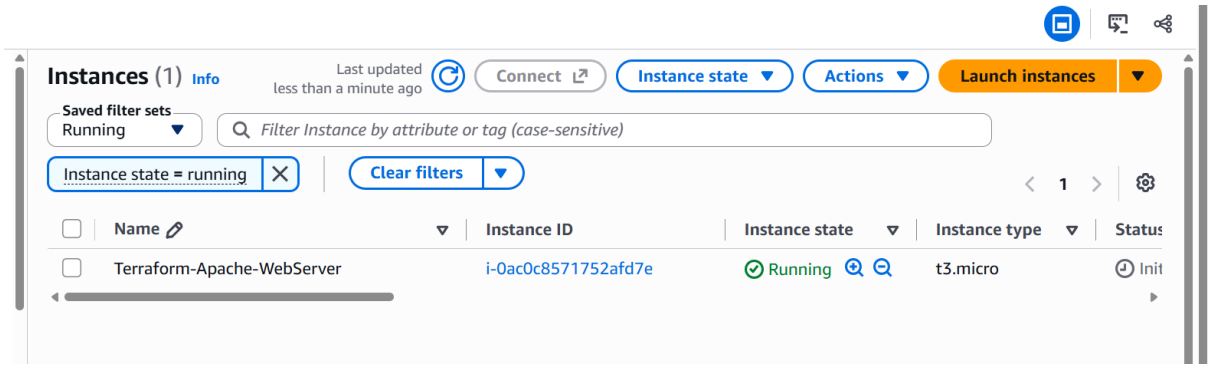
value = "http://${aws_instance.webserver.public_ip}"

}

Result: The EC2 instance and security group were created successfully using Terraform. Apache HTTP Server was installed through the EC2 user data script, and the custom webpage was accessed and verified successfully in the browser.









4. Set up S3 as backend for task 3.

Implementation Steps

1. Used the existing project folder C:\terraform-05\task-3.
2. Added AWS S3 bucket resources to the existing Terraform configuration.

3. Created an S3 bucket named terraform-05-state-poojith-20261101-956035085469.
4. Enabled versioning on the S3 bucket to maintain previous versions of the Terraform state file.
5. Configured server-side encryption using the AES256 algorithm.
6. Enabled all S3 public access block settings to prevent public access to the bucket.
7. Executed terraform plan to review the S3 bucket configuration.
8. Executed terraform apply to create the S3 bucket and its configurations.
9. Removed the temporary S3 resource blocks from main.tf after creating the bucket.
10. Added an S3 backend block to the Terraform configuration to store the state file remotely.
11. Created a backup of the existing local Terraform state file.
12. Executed terraform init -migrate-state to migrate the existing local state file into the S3 backend.
13. Confirmed the migration when Terraform prompted for permission to copy the existing state.
14. Verified the migrated state using terraform state list.
15. Identified that the S3 bucket resources were still present in the migrated state, although their resource blocks had been removed from the configuration.
16. Used terraform state rm to remove those four S3 resource entries from Terraform tracking without deleting the actual AWS bucket.
17. Executed terraform plan and verified that no infrastructure changes were required.

Code – S3 bucket resources

```
resource "aws_s3_bucket" "terraform_state" {
 bucket = "terraform-05-state-poojith-20261101-956035085469"


 tags = {
  Name = "Terraform State Bucket"
```

```
 }
}

resource "aws_s3_bucket_versioning" "terraform_state" {
 bucket = aws_s3_bucket.terraform_state.id

 versioning_configuration {
  status = "Enabled"
 }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "terraform_state" {
 bucket = aws_s3_bucket.terraform_state.id

 rule {
  apply_server_side_encryption_by_default {
   sse_algorithm = "AES256"
  }
 }
}

resource "aws_s3_bucket_public_access_block" "terraform_state" {
 bucket = aws_s3_bucket.terraform_state.id

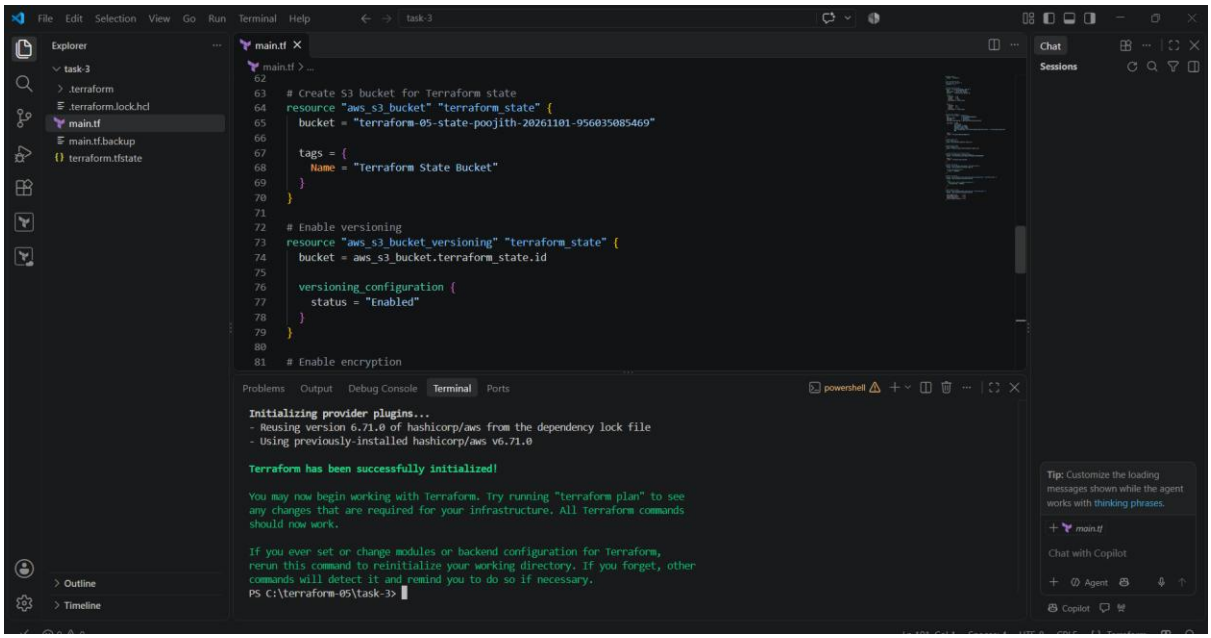
 block_public_acls      = true
 block_public_policy    = true
 ignore_public_acls     = true
 restrict_public_buckets = true
```

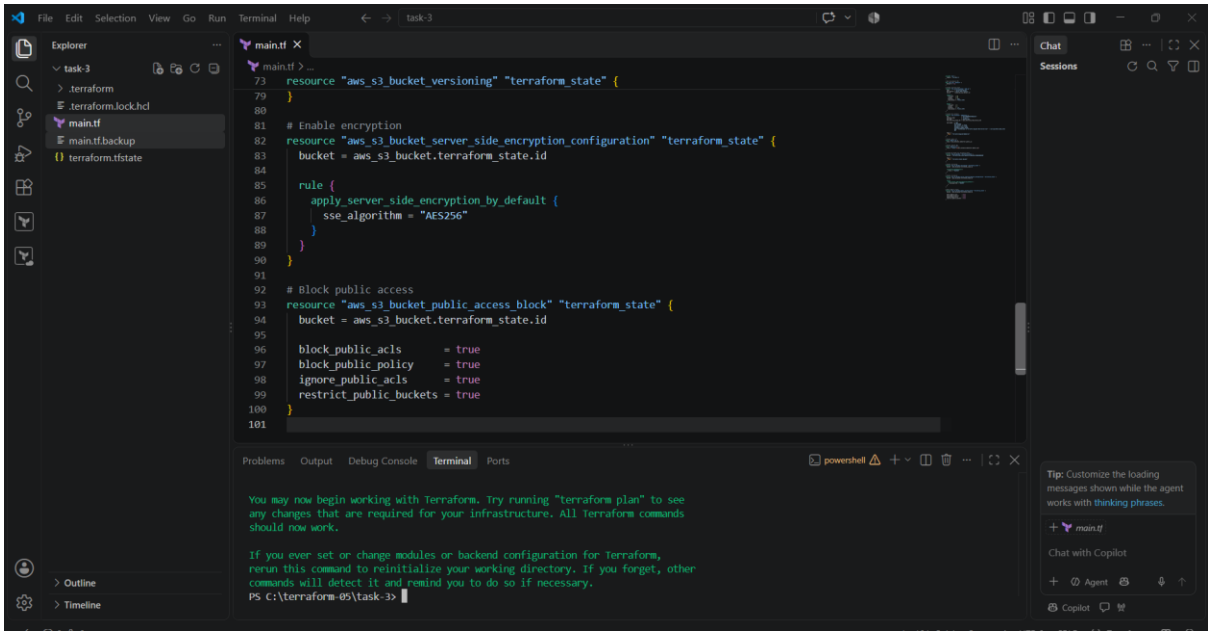
}

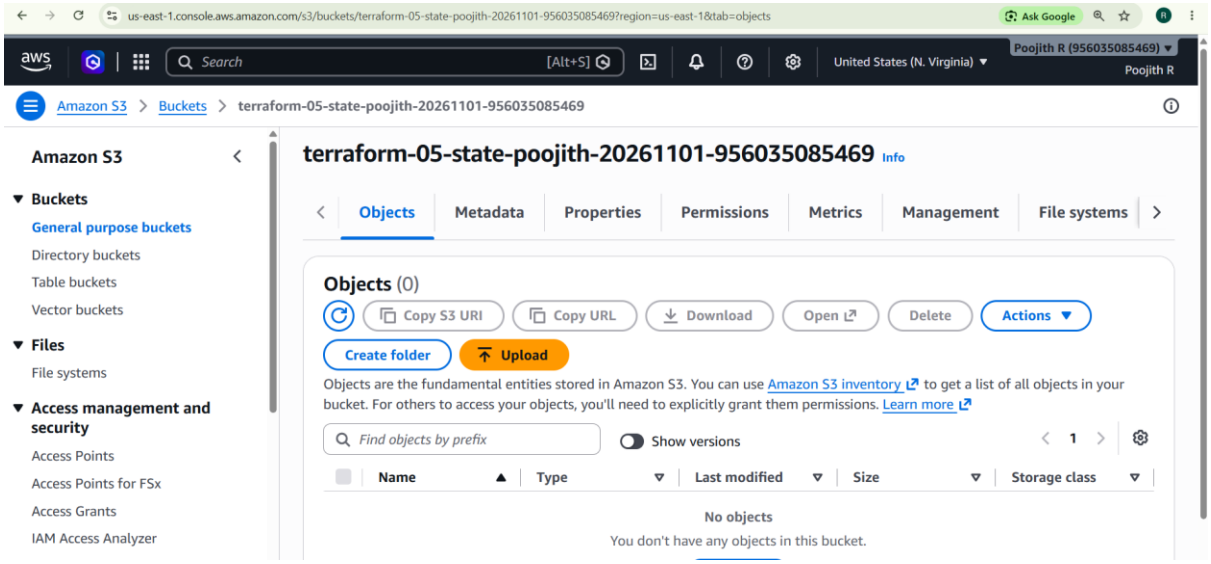
Code – S3 backend configuration

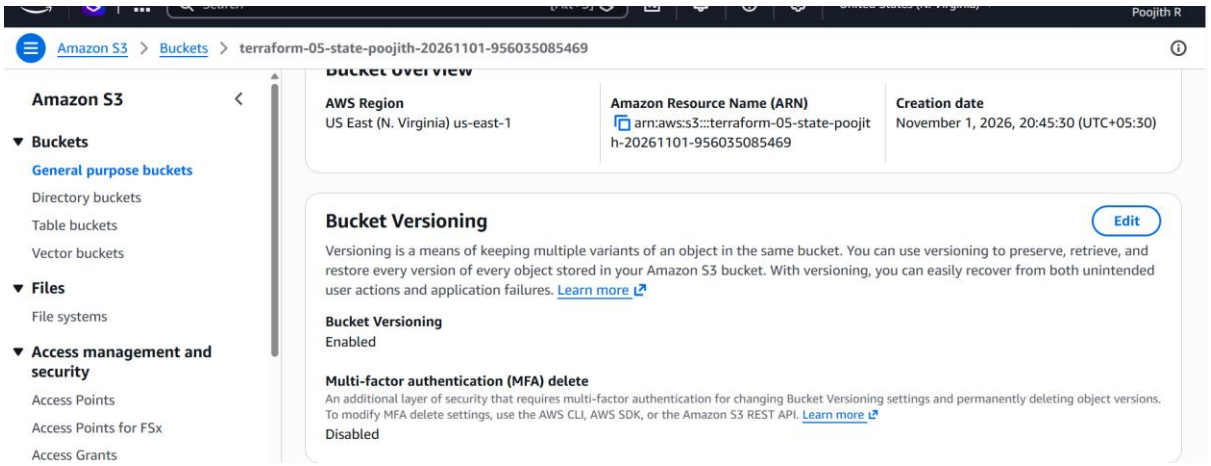
```
terraform {
  backend "s3" {
    bucket  = "terraform-05-state-poojith-20261101-956035085469"
    key     = "ec2-httpd/terraform.tfstate"
    region  = "us-east-1"
    encrypt = true
  }
}
```

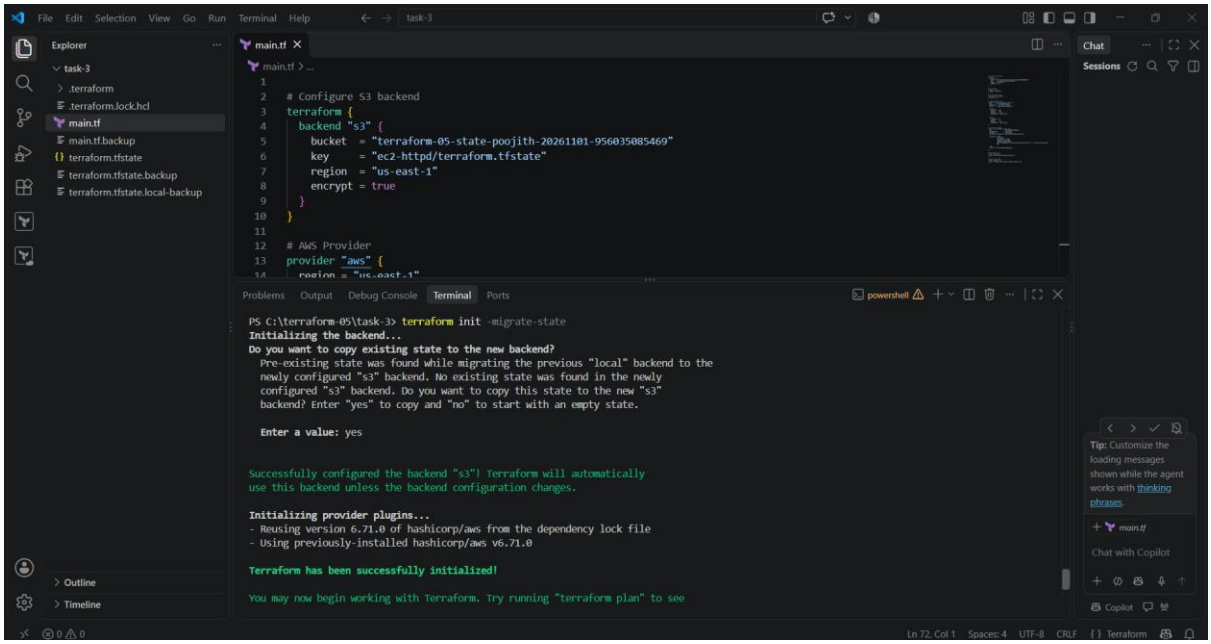
Result: Successfully created the S3 bucket, enabled versioning and encryption, and migrated the existing Terraform state file from the local machine to the S3 remote backend. Verified that the EC2 instance and security group remained intact and that Terraform reported no changes.

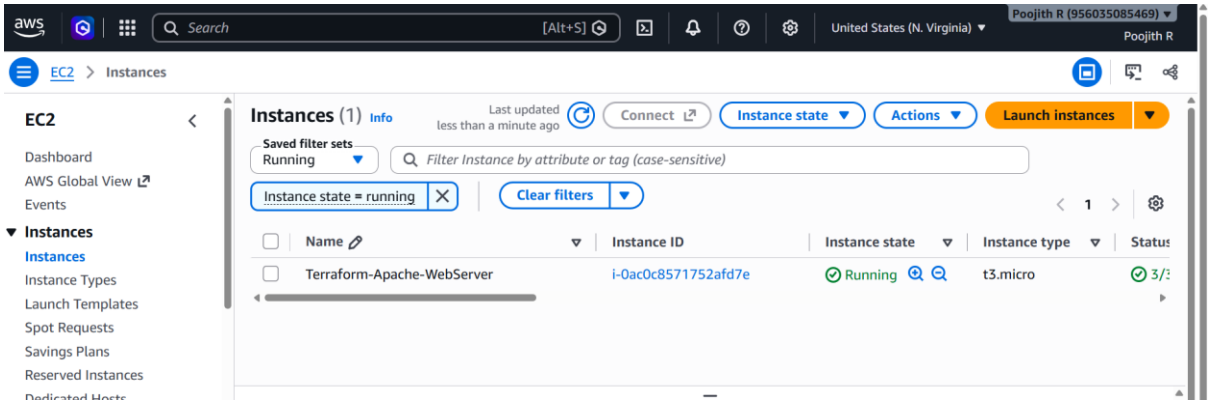


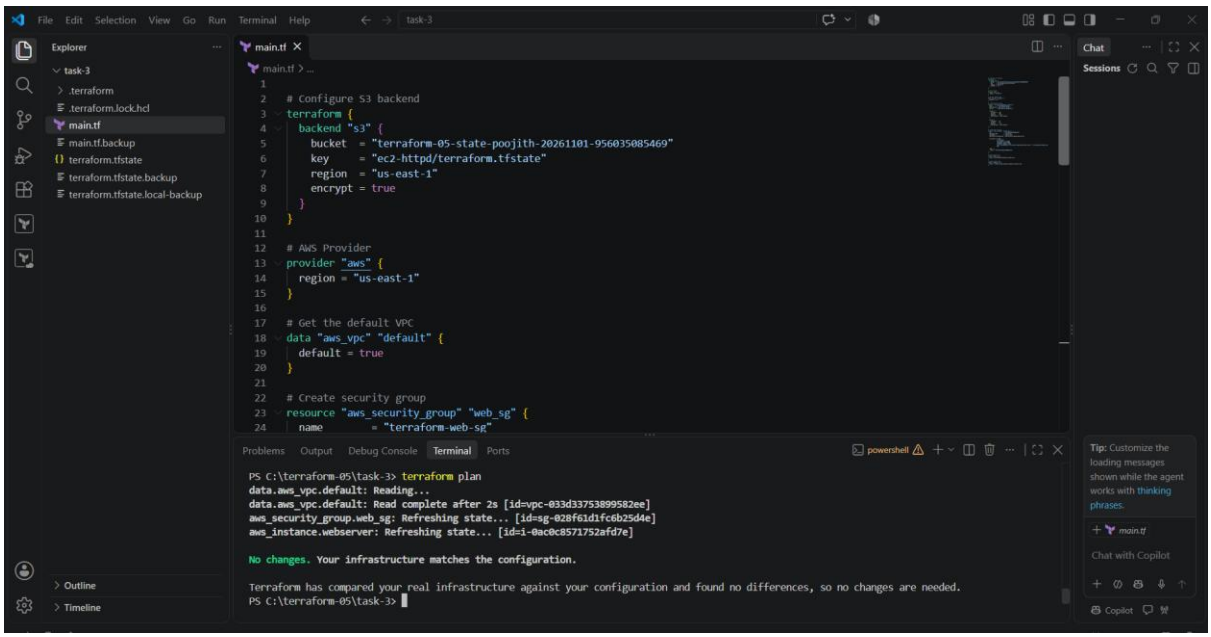

main.tf
resource "aws_s3_bucket_versioning" "terraform_state" {
}
Enable encryption
resource "aws_s3_bucket_server_side_encryption_configuration" "terraform_state" {
bucket = aws_s3_bucket.terraform_state.id
rule {
apply_server_side_encryption_by_default {
sse_algorithm = "AES256"
}
}
}
Block public access
resource "aws_s3_bucket_public_access_block" "terraform_state" {
bucket = aws_s3_bucket.terraform_state.id
block_public_acls = true
block_public_policy = true
ignore_public_acls = true
restrict_public_buckets = true
}
You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.
If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.
PS C:\terraform-05\task-3>

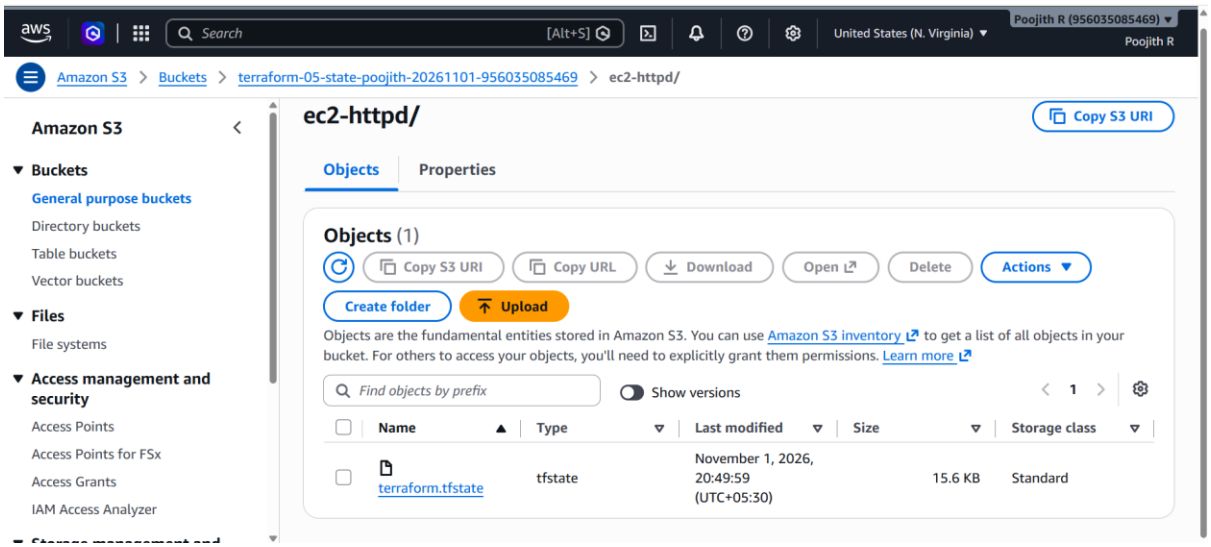

terraform-05-state-poojith-20261101-956035085469 Info
Objects
Metadata
Properties
Permissions
Metrics
Management
File systems
Objects (0)
Copy S3 URI
Copy URL
Download
Open
Delete
Actions
Create folder
Upload
Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more
Find objects by prefix
Show versions
Name
Type
Last modified
Size
Storage class
No objects
You don't have any objects in this bucket.


Bucket Versioning
Edit
Versioning is a means of keeping multiple variants of an object in the same bucket. You can use versioning to preserve, retrieve, and restore every version of every object stored in your Amazon S3 bucket. With versioning, you can easily recover from both unintended user actions and application failures. Learn more
Bucket Versioning
Enabled
Multi-factor authentication (MFA) delete
An additional layer of security that requires multi-factor authentication for changing Bucket Versioning settings and permanently deleting object versions. To modify MFA delete settings, use the AWS CLI, AWS SDK, or the Amazon S3 REST API. Learn more
Disabled


main.tf
Configure S3 backend
terraform {
backend "s3" {
bucket = "terraform-05-state-poojith-20261101-956035085469"
key = "ec2-httpd/terraform.tfstate"
region = "us-east-1"
encrypt = true
}
}
AWS Provider
provider "aws" {
PS C:\terraform-05\task-3> terraform init -migrate-state
Initializing the backend...
Do you want to copy existing state to the new backend?
Pre-existing state was found while migrating the previous "local" backend to the
newly configured "s3" backend. No existing state was found in the newly
configured "s3" backend. Do you want to copy this state to the new "s3"
backend? Enter "yes" to copy and "no" to start with an empty state.
Enter a value: yes
Successfully configured the backend "s3"! Terraform will automatically
use this backend unless the backend configuration changes.
Initializing provider plugins...
- Reusing version 6.71.0 of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.71.0
Terraform has been successfully initialized!
You may now begin working with Terraform. Try running "terraform plan" to see


Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
EC2 > Instances
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Instances (1) Info
Last updated less than a minute ago
Connect
Instance state
Actions
Launch instances
Saved filter sets
Running
Filter Instance by attribute or tag (case-sensitive)
Instance state = running
Clear filters
Name
Instance ID
Instance state
Instance type
Terraform-Apache-WebServer
i-0ac0c8571752afd7e
Running
t3.micro


main.tf
Configure S3 backend
terraform {
backend "s3" {
bucket = "terraform-05-state-poojith-20261101-956035085469"
key = "ec2-httpd/terraform.tfstate"
region = "us-east-1"
encrypt = true
}
}
AWS Provider
provider "aws" {
region = "us-east-1"
}
Get the default VPC
data "aws_vpc" "default" {
default = true
}
Create security group
resource "aws_security_group" "web_sg" {
name = "terraform-web-sg"
PS C:\terraform-05\task-3> terraform plan
data.aws_vpc.default: Reading...
data.aws_vpc.default: Read complete after 2s [id=vpc-033d3375899582ee]
aws_security_group.web_sg: Refreshing state... [id=sg-028f61d1fc6b25d4e]
aws_instance.webserver: Refreshing state... [id=i-0ac0c8571752afd7e]
No changes. Your infrastructure matches the configuration.
Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.
PS C:\terraform-05\task-3>



5. Set up DynamoDB locking for task 3.

Implementation Steps

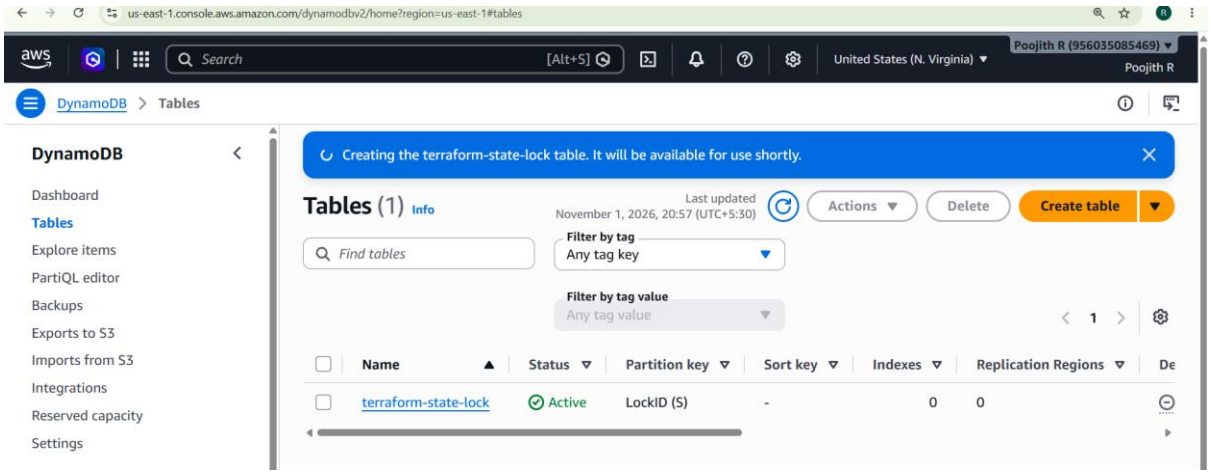
1. Opened the AWS Management Console and navigated to the DynamoDB service.
2. Selected the us-east-1 region.
3. Created a DynamoDB table named terraform-state-lock.
4. Configured the partition key as LockID with the data type String.
5. Selected DynamoDB Standard as the table class.
6. Selected On-demand capacity mode.
7. Created the DynamoDB table and waited until its status changed to Active.
8. Opened the existing Terraform project folder C:\terraform-05\task-3.
9. Updated the existing S3 backend configuration in main.tf by adding the DynamoDB table name.
10. Executed terraform init -reconfigure to reinitialize Terraform with the updated backend configuration.
11. Verified that Terraform successfully configured the S3 backend with DynamoDB locking.
12. Executed terraform state list to verify that the EC2 instance and security group were still present in the remote state.
13. Executed terraform plan to verify the infrastructure configuration.

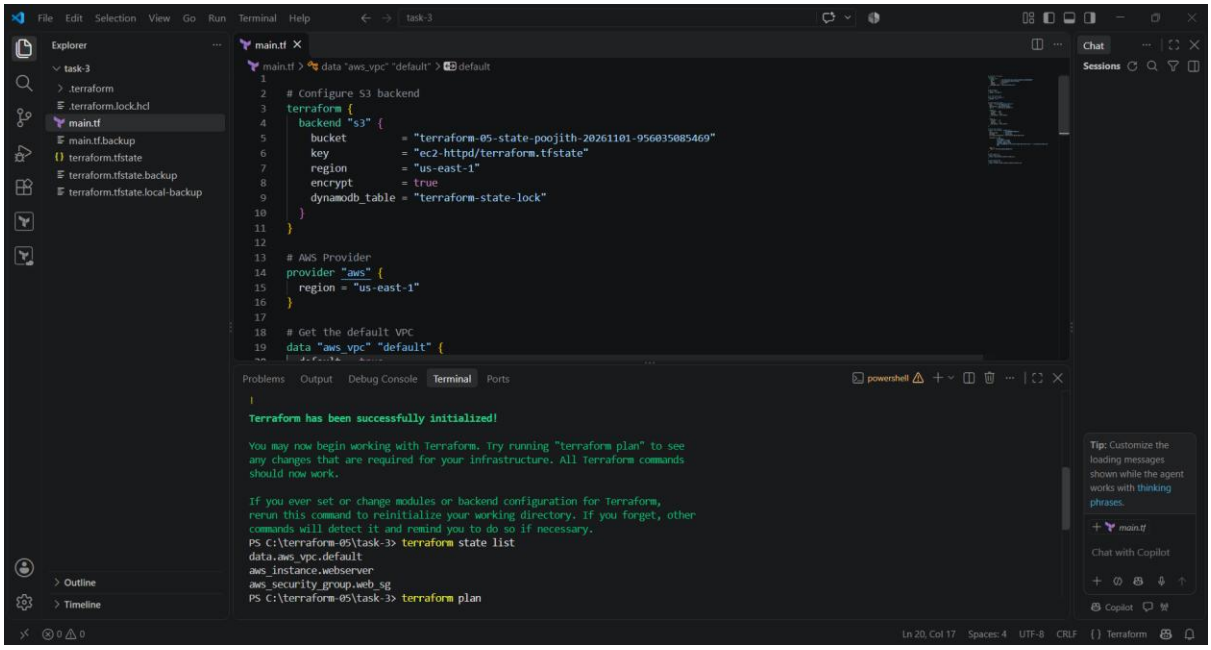
14. Observed Terraform acquiring the state lock before executing the plan.
15. Observed Terraform releasing the state lock after completing the plan.
16. Verified that the plan showed no infrastructure changes.

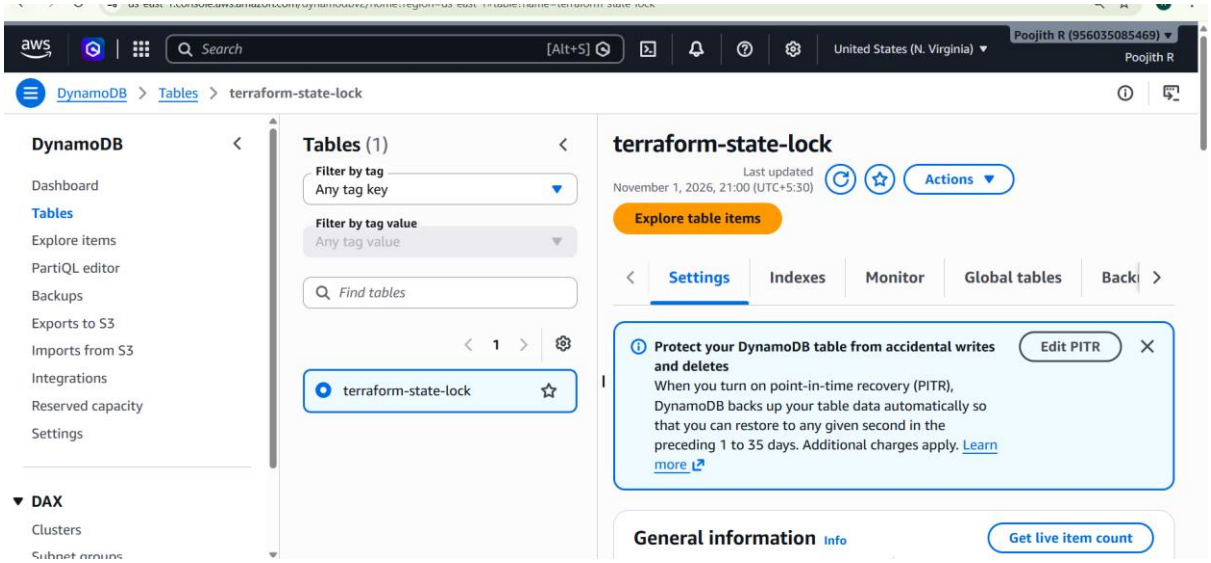
Code – Updated S3 backend with DynamoDB

```
terraform {
 backend "s3" {
  bucket       = "terraform-05-state-poojith-20261101-956035085469"
  key          = "ec2-httpd/terraform.tfstate"
  region       = "us-east-1"
  encrypt      = true
  dynamodb_table = "terraform-state-lock"
 }
}
```

Result: Successfully created the DynamoDB table and configured it with the existing S3 remote backend. Verified that Terraform acquired and released the state lock during the plan operation without making any infrastructure changes.




File Edit Selection View Go Run Terminal Help
task-3
Explorer
task-3
.terraform
.terraform.lock.hcl
main.tf
main.tf.backup
terraform.tfstate
terraform.tfstate.backup
terraform.tfstate.local-backup
Outline
Timeline
main.tf
main.tf > data "aws_vpc" "default" > default
Configure S3 backend
terraform {
backend "s3" {
bucket = "terraform-05-state-poojith-20261101-956035085469"
key = "ec2-httpd/terraform.tfstate"
region = "us-east-1"
encrypt = true
dynamodb_table = "terraform-state-lock"
}
}
AWS Provider
provider "aws" {
region = "us-east-1"
}
Get the default VPC
data "aws_vpc" "default" {
Problems Output Debug Console Terminal Ports
powershell
Terraform has been successfully initialized!
You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.
If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\terraform-05\task-3> terraform state list
data.aws_vpc.default
aws_instance.webserver
aws_security_group.web_sg
PS C:\terraform-05\task-3> terraform plan
Chat
Sessions
Tip: Customize the loading messages shown while the agent works with thinking phrases.
main.tf
Chat with Copilot
Copilot
Ln 20, Col 17 Spaces: 4 UTF-8 CRLF { } Terraform


aws
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
DynamoDB > Tables > terraform-state-lock
DynamoDB
Dashboard
Tables
Explore items
PartiQL editor
Backups
Exports to S3
Imports from S3
Integrations
Reserved capacity
Settings
DAX
Clusters
Subnet groups
Tables (1)
Filter by tag
Any tag key
Filter by tag value
Any tag value
Find tables
1
terraform-state-lock
terraform-state-lock
Last updated
November 1, 2026, 21:00 (UTC+5:30)
Actions
Explore table items
Settings
Indexes
Monitor
Global tables
Back
Protect your DynamoDB table from accidental writes and deletes
When you turn on point-in-time recovery (PITR), DynamoDB backs up your table data automatically so that you can restore to any given second in the preceding 1 to 35 days. Additional charges apply. Learn more
Edit PITR
General information Info
Get live item count